

TDMA SCHEDULE PLANNER AND EMANE INTEGRATION

Technical Documentation

Vaan Megam Networks - Wireless Developer Internship

Candidate	Thamarai Kannan P
Part 1	16-node TDMA scheduler - 9-slot conflict-free schedule
Part 2	Python-to-EMANE TDMA integration - 16-NEM validation
Final enforcement test	Allowed: 17 upstream packets Blocked: 0 PASS

GitHub: <https://github.com/KANNAN-7777/tdma-scheduler>

This report records the design decisions, implementation, final values, and runtime evidence used for the submission.

1. Project Objective

The project implements a centralized TDMA schedule planner for a 16-node wireless network and connects the generated schedule to EMANE for runtime validation. The scheduler starts from static node coordinates, builds a communication graph, models direct and two-hop conflicts with a distance-2 conflict graph, applies graph-coloring heuristics, and verifies the final schedule.

The second part converts the Node-to-Slot result into a native EMANE TDMA schedule, runs it on a 16-NEM platform, and checks that packet behavior changes when the transmission opportunity is present or removed.

2. Part 1 - Design Process

2.1 Communication Graph

Each wireless node is represented as a graph vertex. An undirected communication edge is created when the Euclidean distance between two nodes is less than or equal to 500 m.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

2.2 Distance-2 Conflict Graph

The communication graph is converted into a distance-2 conflict graph. Nodes cannot share a TDMA slot when they are directly connected or when they are separated by a two-hop path. This models the direct and hidden-terminal style conflicts required by the problem statement.

2.3 Graph Coloring and Optimization

Each color is treated as a TDMA slot. Three greedy coloring strategies were evaluated: Largest First, Smallest Last, and Saturation Largest First. The smallest slot count among the tested strategies was selected. A local improvement pass then tries to move nodes into lower existing slots without creating a conflict.

The result is treated as a heuristic solution rather than a proof of global minimum coloring.

2.4 Verification

The verifier checks every edge in the distance-2 conflict graph and requires different slots at both endpoints. The final schedule passed the verification check.

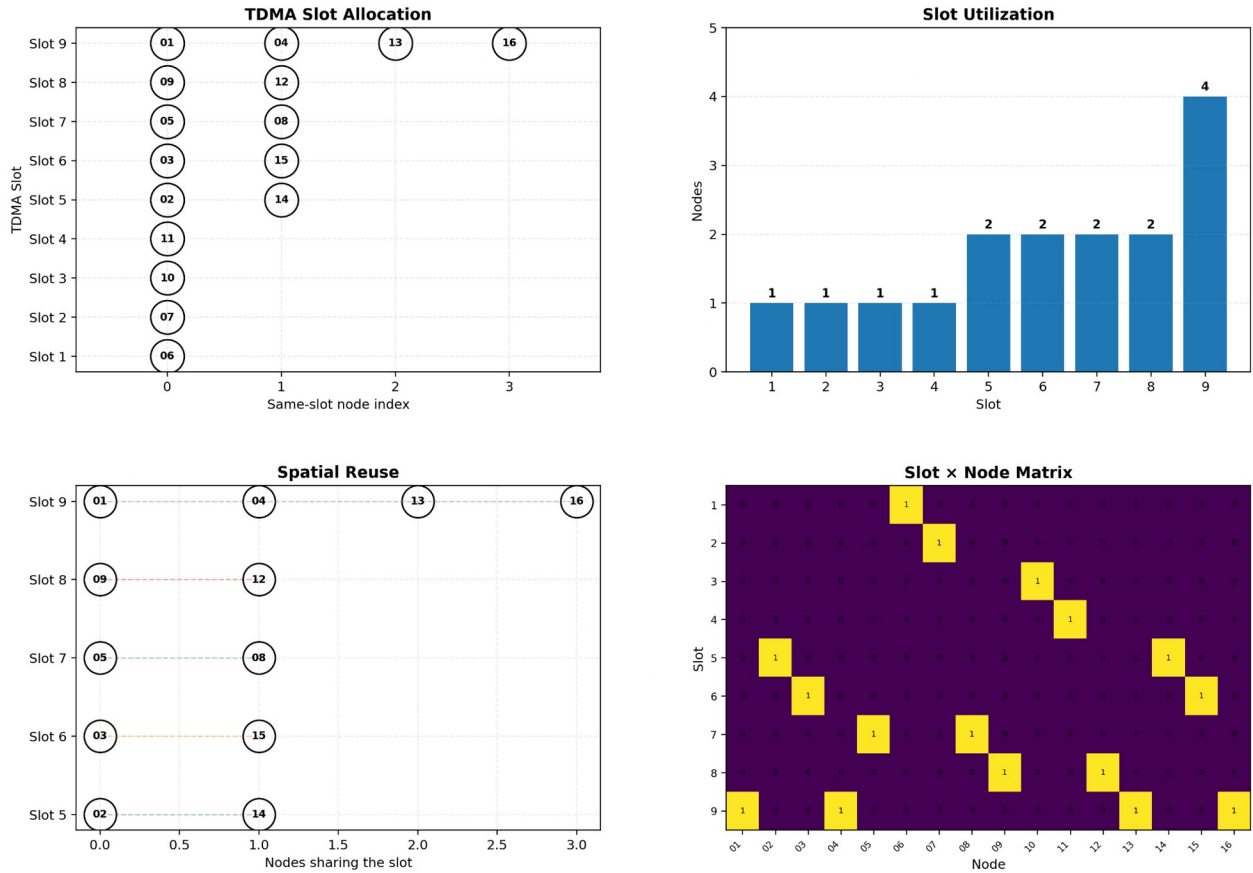
3. Part 1 - Final Values

Metric	Final value
Nodes	16
Communication range	500 m
Direct communication links	42
Distance-2 conflict links	90
Coloring strategies tested	3
Selected TDMA slots	9
Reused slots	5
Maximum nodes in one slot	4
Schedule verification	PASS

All three tested coloring strategies produced a 9-slot result for the supplied topology. The final report does not claim that 9 is the theoretical minimum.

4. Final Schedule and Spatial Reuse

TDMA Part 1 — 16 Node Schedule Analysis



Verified result: 16 nodes | 9 TDMA slots | 5 reused slots | maximum 4 nodes in one slot | conflict verification PASS

Figure 1. Final TDMA schedule analysis for the tested 16-node network.

4.1 Node-to-Slot Mapping

Node	Slot	Node	Slot
Node_01	9	Node_09	8
Node_02	5	Node_10	3
Node_03	6	Node_11	4
Node_04	9	Node_12	8
Node_05	7	Node_13	9
Node_06	1	Node_14	5
Node_07	2	Node_15	6
Node_08	7	Node_16	9

4.2 Reused Slots

Slot	Nodes sharing the slot
5	Node_02, Node_14
6	Node_03, Node_15
7	Node_05, Node_08
8	Node_09, Node_12
9	Node_01, Node_04, Node_13, Node_16

The nodes listed under the same slot are non-conflicting in the distance-2 conflict graph, so the slot can be reused.

4.3 Slot x Node Matrix

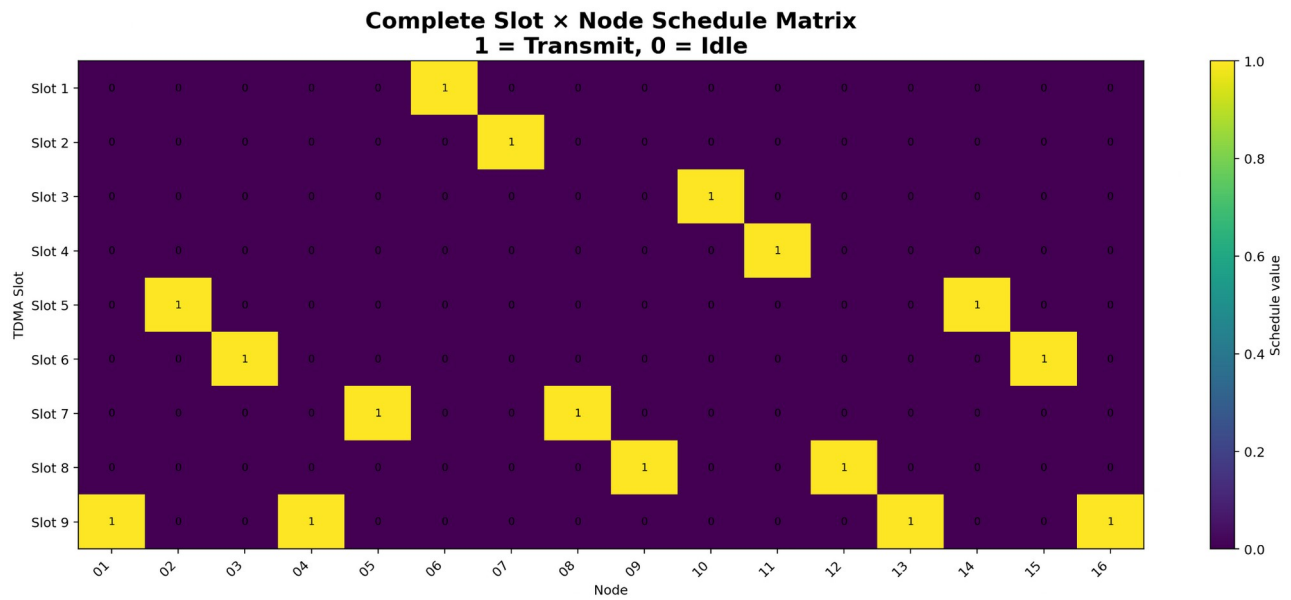


Figure 2. Complete Slot x Node binary schedule matrix (1 = transmit, 0 = idle).

4.4 Slot Utilization

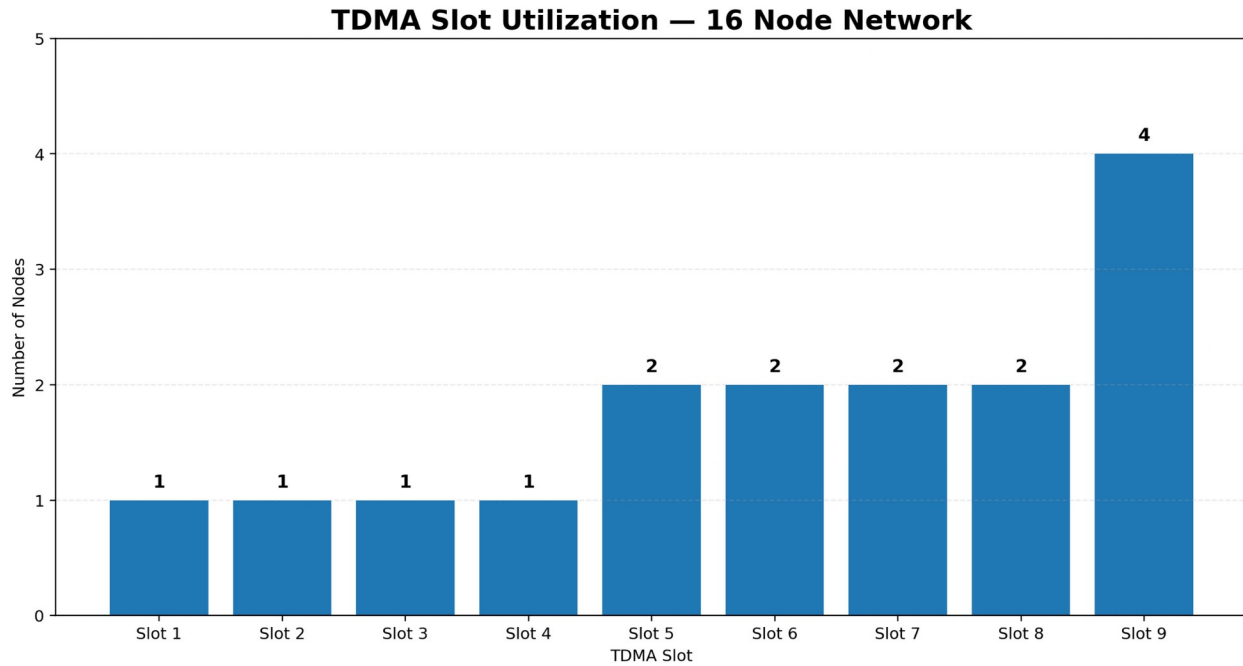


Figure 3. Number of nodes assigned to each TDMA slot.

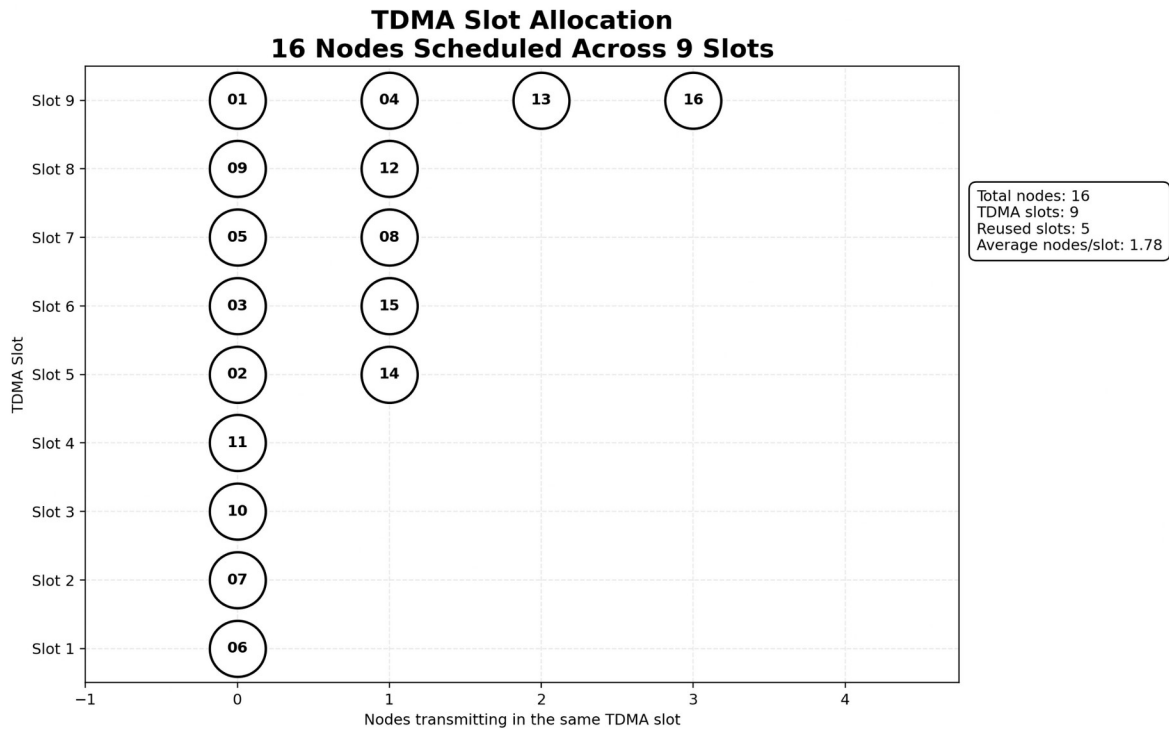


Figure 4. Node allocation across the nine TDMA slots.

5. Part 2 - EMANE Integration

The Part 2 bridge keeps the scheduler independent from EMANE. The Node-to-Slot result produced by Part 1 is converted into a native EMANE TDMA schedule XML and then published to the running TDMA MACs.

Part 1 scheduler

- > Node-to-Slot mapping
- > generate_emane_schedule.py
- > emane_tdma_schedule.xml
- > emaneevent-tdmaschedule
- > 16-NEM EMANE runtime

5.1 Native TDMA Parameters

Parameter	Value
Slot duration	1 ms
Frames	1
TDMA slots	9
Frequency	2.4 GHz
Bandwidth	1 MHz
Data rate	1 Mbit/s
Service class	0

5.2 Runtime Validation

Check	Observed result
EMANE platform	16 NEMs running
Schedule acceptance	scheduleAcceptFull = 1 for NEM 1-16
Frame index rejects	0
Other schedule rejects	0
Slot index rejects	0
Update-before-full rejects	0

The regenerated schedule was accepted by all 16 NEMs, and all tested schedule rejection counters remained zero.

6. Schedule Enforcement Test

A controlled two-case EMANE test was used to check packet behavior against the Part 1 schedule. Node_01 is assigned Slot 9 in Part 1, which maps to EMANE zero-based slot index 8.

6.1 Allowed Case

NEM 1 was given a TX opportunity in the corresponding TDMA slot and 50 raw Ethernet test frames were generated toward NEM 2. NEM 2 processed 17 upstream test packets.

6.2 Blocked Case

The same test was repeated in a clean EMANE run after changing NEM 1's corresponding slot from TX to RX. This removed NEM 1's TX opportunity. The same 50 frames were generated, and NEM 2 processed 0 upstream test packets.

Enforcement check	Result
Allowed schedule - traffic processed	17 upstream packets
Blocked schedule - traffic processed	0 upstream packets
Allowed case	PASS
Blocked case	PASS
Overall schedule enforcement	PASS

The test demonstrates that packet behavior changed with the TDMA schedule derived from the Part 1 Node-to-Slot assignment. The values above are the measured MAC counters from the clean allowed and blocked runs.

7. Implementation Structure

File	Purpose
main.py	End-to-end Part 1 scheduler execution
graph_builder.py	Communication and distance-2 conflict graph
coloring.py	Graph-coloring strategies
scheduler.py	Slot numbering and local optimization
verifier.py	Conflict verification
generate_emane_schedule.py	Part 1 Node-to-Slot to EMANE TDMA XML bridge
send_tdma_test.py	Raw Ethernet packet test
validate_emane.py	Generated XML validation
Part2_Enforcement_Evidence.txt	Allowed/blocked enforcement evidence

8. Final Result

Part 1 produced a conflict-free 9-slot schedule for the tested 16-node topology, with five reused slots and a maximum of four nodes sharing a slot. The distance-2 verifier reported PASS.

Part 2 converted the Python schedule into a native EMANE TDMA schedule, loaded it into a 16-NEM runtime, and achieved full schedule acceptance with zero tested schedule-rejection counters. The controlled enforcement test then showed 17 upstream packets in the allowed case and 0 in the blocked case, with an overall enforcement result of PASS.

9. Submission Contents

- Python scheduler and test source code
- Input and validation test cases
- EMANE configuration and generated schedule XML
- Part 1 and Part 2 runtime evidence files
- Technical documentation PDF
- Technical presentation PPT

GitHub repository: <https://github.com/KANNAN-7777/tdma-scheduler>

10. Technical Defence Points

- Why distance-2 conflicts are required for one-hop and two-hop interference avoidance.
- Why graph coloring maps naturally to TDMA slot allocation.
- Why the selected 9 slots are a heuristic result rather than a proven minimum.
- How slot reuse is safe when shared-slot nodes are non-conflicting.
- How the Python bridge maps Node-to-Slot values into EMANE TDMA slots.
- What scheduleAcceptFull and scheduleReject counters prove.
- Why the allowed and blocked packet tests demonstrate schedule-controlled packet behavior.